Introdução aos AFDs

Módulo 6 – Ciência da Computação

Semana 9

Fillipe Resina

Professor de Programação

fillipe.resina@prof.inteli.edu.br

Parte 1: Otimização Combinatória

- Visão Geral de Otimização Combinatória
- Programação Linear
- Programação Dinâmica
- Busca Local
- Programação Inteira Mista
- Algoritmos Gulosos
- Programação com Restrições
- Backtracking
- Modelagem

Parte 2: Teoria da Computação e Linguagens Formais

- P vs NP e NP-Completude
- Introdução aos autômatos finitos determinísticos e não-determinísticos

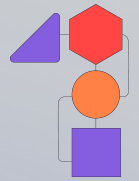
ENCONTROS DO MÓDULO

Objetivos da instrução

- Introduzir conceitos de linguagens formais;
- Introduzir o conceito de máquina de estados;
- Entender limitações das linguagens regulares e dos AFDs;
- Modelagem de máquinas abstratas que reconhecem linguagens;

inteli

inteli



Conceitos iniciais

O que é um autômato finito determinístico (AFD)?

- **Modelo de computador simplificado**
  - Sistemas de **estados finitos** são usados para explicar o funcionamento de máquinas ou sistemas autônomos utilizados em diversas aplicações:
    - *Portas automáticas, elevadores, máquinas de refrigerante, etc*

  - Modelo de computador com **memória** extremamente **limitada**, determinada pelo número de estados (veremos esse conceito a seguir)

O que é um autômato finito determinístico (AFD)?

- **Modelo de computador simplificado**
  - Sistemas de **estados finitos** são usados para explicar o funcionamento de máquinas ou sistemas autônomos utilizados em diversas aplicações:
    - *Portas automáticas, elevadores, máquinas de refrigerante, etc*
  - Modelo de computador com **memória** extremamente **limitada**, determinada pelo número de estados (veremos esse conceito a seguir)
- **É possível modelar matematicamente um autômato finito**
  - Os problemas são apresentados como linguagens (padrões) que o autômato finito deve reconhecer, com duas respostas possíveis: aceita ou rejeita

inteli

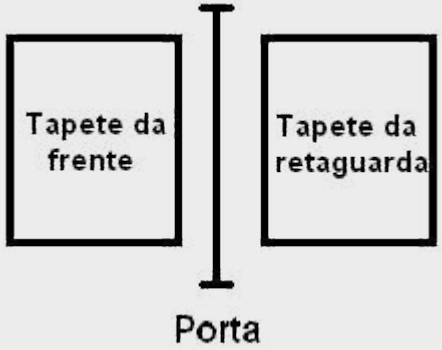
O que é um autômato finito determinístico (AFD)?

- **Modelo de computador simplificado**
  - Sistemas de **estados finitos** são usados para explicar o funcionamento de máquinas ou sistemas autônomos utilizados em diversas aplicações:
    - *Portas automáticas, elevadores, máquinas de refrigerante, etc*
  - Modelo de computador com **memória** extremamente **limitada**, determinada pelo número de estados (veremos esse conceito a seguir)
- **É possível modelar matematicamente um autômato finito**
  - Os problemas são apresentados como linguagens (padrões) que o autômato finito deve reconhecer, com duas respostas possíveis: aceita ou rejeita
- **Importância e aplicações**: reconhecer linguagens regulares
  - Construção de analisadores léxicos em compiladores
  - Expressões regulares
  - Entender outras máquinas abstratas mais complexas



inteli

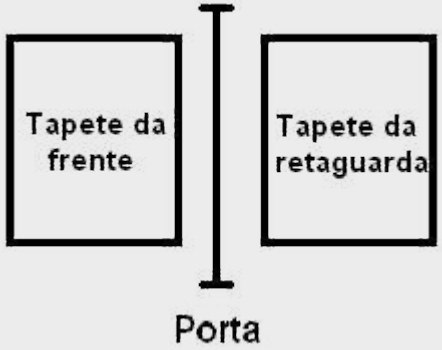
Um autômato simples: “controlador de porta automática”



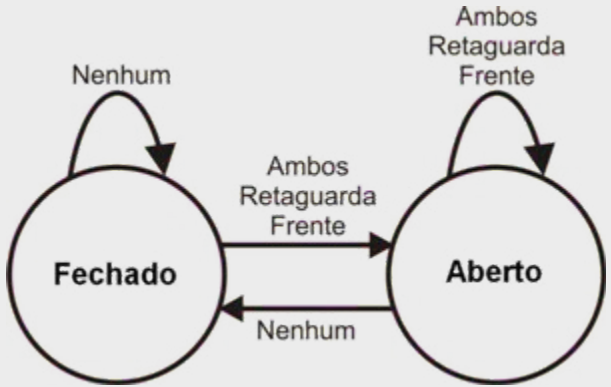
- Temos 4 estímulos diferentes:
  - **Frente**: há uma pessoa está no tapete da frente;
  - **Retaguarda**: há uma pessoa está no tapete de dentro;
  - **Ambos**: há pessoas sobre os dois tapetes;
  - **Nenhum**: não há ninguém sobre os tapetes;

inteli

Um autômato simples: “controlador de porta automática”



- Temos 4 estímulos diferentes:
  - **Frente**: há uma pessoa está no tapete da frente;
  - **Retaguarda**: há uma pessoa está no tapete de dentro;
  - **Ambos**: há pessoas sobre os dois tapetes;
  - **Nenhum**: não há ninguém sobre os tapetes;



- Para cada estímulo, o controlador vai responder de uma forma diferente, realizando as transições de um estado para outro.

Um autômato simples: “controlador de porta automática”

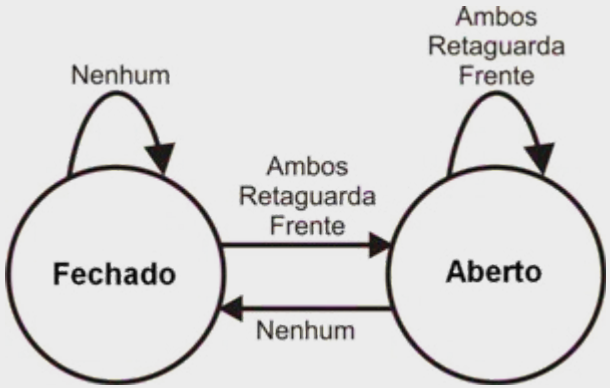


Tabela de estados de transição:

	Nenhum	Frente	Retaguarda	Ambos
Aberto				
Fechado				

inteli

Um autômato simples: “controlador de porta automática”

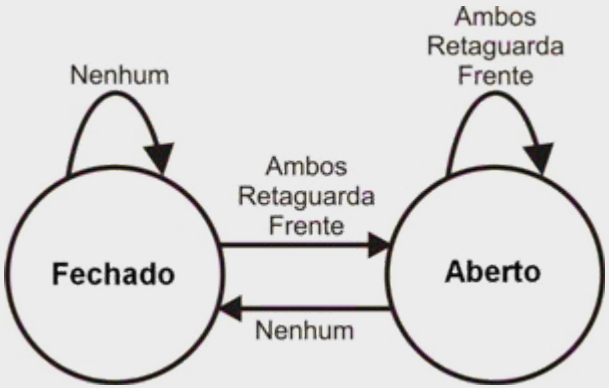


Tabela de estados de transição:

	Nenhum	Frente	Retaguarda	Ambos
Aberto	Fechado	Aberto	Aberto	Aberto
Fechado	Fechado	Aberto	Aberto	Aberto

inteli

Um autômato simples: “controlador de porta automática”

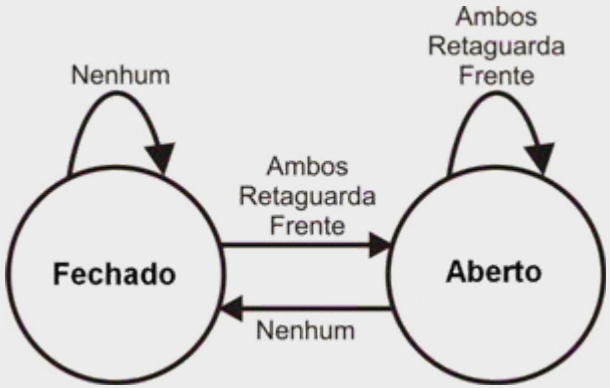


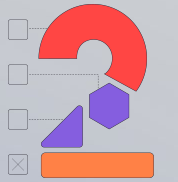
Tabela de estados de transição:

	Nenhum	Frente	Retaguarda	Ambos
Aberto	Fechado	Aberto	Aberto	Aberto
Fechado	Fechado	Aberto	Aberto	Aberto

Podemos representar a tabela acima como uma **função** (representada pela letra grega delta **δ**) que, dada uma combinação entre um **estado** e um sinal de entrada (**alfabeto**), resulta em outro **estado**:

$$\delta : \underbrace{\text{Estado} \times \text{Alfabeto}}_{\text{Entrada da função } \delta} \rightarrow \underbrace{\text{Estado}}_{\text{Saída da função } \delta}$$

inteli



Alfabetos e cadeias

Revisão: alfabetos e cadeias

- Para simplificar a notação, utilizamos o conceito de alfabetos e cadeias para representar um "sinal de entrada":
  - **Alfabeto**: conjunto finito de elementos (símbolos), denotado geralmente por Σ;
    - Exemplos:

$$\Sigma_1 = \{a, b\}, \Sigma_2 = \{0, 1\}, \Sigma_3 = \{a, b, c, \ldots, x, y, z\}, \Sigma_4 = \{\#, =, +, -, *, /, 0, 1\}$$

inteli

Revisão: alfabetos e cadeias

- Para simplificar a notação, utilizamos o conceito de alfabetos e cadeias para representar um "sinal de entrada":
  - **Alfabeto**: conjunto finito de elementos (símbolos), denotado geralmente por Σ;
    - Exemplos:

$$\Sigma_1 = \{a, b\}, \Sigma_2 = \{0, 1\}, \Sigma_3 = \{a, b, c, \ldots, x, y, z\}, \Sigma_4 = \{\#, =, +, -, *, /, 0, 1\}$$

  - **Cadeia** (string/palavra): é uma sequência finita de símbolos sobre um alfabeto Σ;
    - Formalmente: $x_1 x_2, \ldots, x_n, \text{ com } x_i \in \Sigma \text{ para } 1 \leq i \leq n$
    - Exemplos:
      - **1100** é uma cadeia sobre o alfabeto Σ = {0, 1}
      - **acaba** é uma cadeia sobre o alfabeto Σ = {a, b, c}
      - **inteli** é uma cadeia sobre o alfabeto Σ = {a, b, ..., z}

inteli

Revisão: alfabetos e cadeias

- Para simplificar a notação, utilizamos o conceito de alfabetos e cadeias para representar um "sinal de entrada":
  - **Alfabeto**: conjunto finito de elementos (símbolos), denotado geralmente por Σ;
    - Exemplos:

$$\Sigma_1 = \{a, b\}, \Sigma_2 = \{0, 1\}, \Sigma_3 = \{a, b, c, \ldots, x, y, z\}, \Sigma_4 = \{\#, =, +, -, *, /, 0, 1\}$$

  - **Cadeia** (string/palavra): é uma sequência finita de símbolos sobre um alfabeto Σ;
    - Formalmente: $x_1 x_2, \ldots, x_n, \text{ com } x_i \in \Sigma \text{ para } 1 \leq i \leq n$
    - Exemplos:
      - **1100** é uma cadeia sobre o alfabeto Σ = {0, 1}
      - **acaba** é uma cadeia sobre o alfabeto Σ = {a, b, c}
      - **inteli** é uma cadeia sobre o alfabeto Σ = {a, b, ..., z}

  - **Comprimento de uma cadeia**
    - Exemplos:

$$|1100| = 4$$

$$|acaba| = 5$$

$$|inteli| = 6$$

inteli

Revisão: alfabetos e cadeias

- Outras definições importantes:
  - **Concatenação**: $\alpha\beta = a_1 a_2 \ldots a_n b_1 b_2 \ldots b_m$
    - Exemplo: se α = 1100 e β = aabb, então: αβ = 1100aabb

inteli

Revisão: alfabetos e cadeias

- Outras definições importantes:
  - **Concatenação**: $\alpha\beta = a_1 a_2 \ldots a_n b_1 b_2 \ldots b_m$
    - Exemplo: se α = 1100 e β = aabb, então: αβ = 1100aabb
  - **Potência de cadeia**: $w^k = \underbrace{ww \ldots w}_{k \text{ vezes}}$
    - Exemplos:
      - se α = 1100, então α^3 = 110011001100
      - se β = aabb, então β^2 = aabbaabb

inteli

Revisão: alfabetos e cadeias

- Outras definições importantes:
  - **Concatenação**: $\alpha\beta = a_1 a_2 \ldots a_n b_1 b_2 \ldots b_m$
    - Exemplo: se α = 1100 e β = aabb, então: αβ = 1100aabb
  - **Potência de cadeia**: $w^k = \underbrace{ww \ldots w}_{k \text{ vezes}}$
    - Exemplos:
      - se α = 1100, então α^3 = 110011001100
      - se β = aabb, então β^2 = aabbaabb
  - **Potência de símbolo**: ajuda a simplificar a notação de cadeias com símbolos repetidos
    - Exemplos:
      - $a^5bc^3a = aaaaabccca$
      - $a^4b^4 = aaaabbbb$

inteli

Revisão: alfabetos e cadeias

- Outras definições importantes:
  - **Concatenação**: $\alpha\beta = a_1 a_2 \ldots a_n b_1 b_2 \ldots b_m$
    - Exemplo: se α = 1100 e β = aabb, então: αβ = 1100aabb
  - **Potência de cadeia**: $w^k = \underbrace{ww \ldots w}_{k \text{ vezes}}$
    - Exemplos:
      - se α = 1100, então α^3 = 110011001100
      - se β = aabb, então β^2 = aabbaabb
  - **Potência de símbolo**: ajuda a simplificar a notação de cadeias com símbolos repetidos
    - Exemplos:
      - $a^5bc^3a = aaaaabccca$
      - $a^4b^4 = aaaabbbb$
  - **Cadeia vazia**: denotada pela letra grega épsilon **ε** (ε é um conceito, não um símbolo!)
    - É a cadeia de comprimento zero, isto é, **|ε| = 0**
    - Dada uma cadeia w sobre um alfabeto Σ, é sempre verdade que:

      wε = εw = w

inteli

Revisão: potência de alfabeto

- Dado um alfabeto Σ, denotamos Σ^k o conjunto de todas as cadeias de comprimento k sobre Σ:
  - Exemplo: neste slide, considere o alfabeto **Σ = {0, 1}**

$\Sigma^0 = \{\varepsilon\}$
$\Sigma^1 = \{0, 1\}$
$\Sigma^2 = \{00, 01, 10, 11\}$
$\Sigma^3 = \{000, 001, 010, 011, 100, 101, 110, 111\}$

inteli

Revisão: potência de alfabeto

- Dado um alfabeto Σ, denotamos Σ^k o conjunto de todas as cadeias de comprimento k sobre Σ:
  - Exemplo: neste slide, considere o alfabeto **Σ = {0, 1}**

    $\Sigma^0 = \{\varepsilon\}$
    $\Sigma^1 = \{0, 1\}$
    $\Sigma^2 = \{00, 01, 10, 11\}$
    $\Sigma^3 = \{000, 001, 010, 011, 100, 101, 110, 111\}$

- **Fecho de Kleene** (também conhecido como operador "estrela"): é o conjunto de todas as cadeias sobre um alfabeto Σ, denotado por **Σ***:

$$\Sigma^* = \Sigma^0 \cup \Sigma^1 \cup \Sigma^2 \cup \Sigma^3 \cup \dots$$

inteli

Revisão: potência de alfabeto

- Dado um alfabeto Σ, denotamos Σ^k o conjunto de todas as cadeias de comprimento k sobre Σ:
  - Exemplo: neste slide, considere o alfabeto **Σ = {0, 1}**

    Σ^0 = {ε}
    Σ^1 = {0, 1}
    Σ^2 = {00, 01, 10, 11}
    Σ^3 = {000, 001, 010, 011, 100, 101, 110, 111}

- **Fecho de Kleene** (também conhecido como operador "estrela"): é o conjunto de todas as cadeias sobre um alfabeto Σ, denotado por **Σ***:

$$\Sigma^* = \Sigma^0 \cup \Sigma^1 \cup \Sigma^2 \cup \Sigma^3 \cup \dots$$

Σ* = {ε, 0, 1, 00, 01, 10, 11, 000, 001, 010, 011, 100, 101, 110, 111, 0000, 0001, 0010, 0011, 0100, ...}

inteli

Revisão: potência de alfabeto

- Dado um alfabeto Σ, denotamos Σ^k o conjunto de todas as cadeias de comprimento k sobre Σ:
  - Exemplo: neste slide, considere o alfabeto **Σ = {0, 1}**

$\Sigma^0 = \{\varepsilon\}$
$\Sigma^1 = \{0, 1\}$
$\Sigma^2 = \{00, 01, 10, 11\}$
$\Sigma^3 = \{000, 001, 010, 011, 100, 101, 110, 111\}$

- **Fecho de Kleene** (também conhecido como operador "estrela"): é o conjunto de todas as cadeias sobre um alfabeto Σ, denotado por **Σ***:

$$\Sigma^* = \Sigma^0 \cup \Sigma^1 \cup \Sigma^2 \cup \Sigma^3 \cup \dots$$

Σ* = {ε, 0, 1, 00, 01, 10, 11, 000, 001, 010, 011, 100, 101, 110, 111, 0000, 0001, 0010, 0011, 0100, ...}

- **Fecho positivo:** denotado por **Σ⁺**, é definido como:

$$\Sigma^+ = \Sigma^1 \cup \Sigma^2 \cup \Sigma^3 \cup \dots$$

inteli

Revisão: potência de alfabeto

- Dado um alfabeto Σ, denotamos Σ^k o conjunto de todas as cadeias de comprimento k sobre Σ:
  - Exemplo: neste slide, considere o alfabeto **Σ = {0, 1}**

$\Sigma^0 = \{\varepsilon\}$
$\Sigma^1 = \{0, 1\}$
$\Sigma^2 = \{00, 01, 10, 11\}$
$\Sigma^3 = \{000, 001, 010, 011, 100, 101, 110, 111\}$

- **Fecho de Kleene** (também conhecido como operador "estrela"): é o conjunto de todas as cadeias sobre um alfabeto Σ, denotado por **Σ***:

$$\Sigma^* = \Sigma^0 \cup \Sigma^1 \cup \Sigma^2 \cup \Sigma^3 \cup \dots$$

Σ* = {ε, 0, 1, 00, 01, 10, 11, 000, 001, 010, 011, 100, 101, 110, 111, 0000, 0001, 0010, 0011, 0100, ...}

- **Fecho positivo:** denotado por **Σ⁺**, é definido como:

$$\Sigma^+ = \Sigma^1 \cup \Sigma^2 \cup \Sigma^3 \cup \dots$$

Σ⁺ = {0, 1, 00, 01, 10, 11, 000, 001, 010, 011, 100, 101, 110, 111, 0000, 0001, 0010, 0011, 0100, ...}

inteli

inteli



Linguagens

Revisão: linguagem

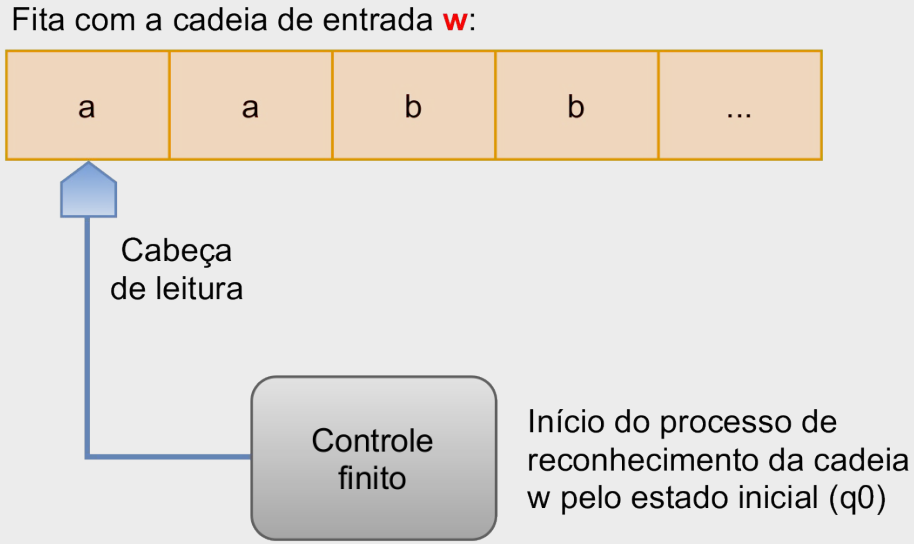
- Uma linguagem L sobre um alfabeto Σ é um subconjunto de Σ*: $L \subseteq \Sigma^*$
  - Ou seja: uma linguagem é um subconjunto de cadeias definidas a partir de um alfabeto!

  - Exemplos:
    - $L_1 = \{00, 01, 10, 11\}$
    - $L_2 = \{w \in \{0,1\}^* \mid w \text{ contém o mesmo número de 0s e 1s}\}$
    - $L_3 = \{a^n b^n \in \{a,b\}^* \mid n \geq 1\}$
    - $L_4 = \{w \in \{0,1,2\}^* \mid \text{o quinto símbolo de } w \text{ é } 2\}$
    - $L_5 = \{1^n 01100^n \in \{0,1\}^* \mid n \geq 0\}$
    - $L_6 = \{w \mid w \text{ é um programa sintaticamente correto em Java}\}$
    - $L_7 = \{\varepsilon\}$
    - $L_8 = \{\ \}$

inteli

Autômato finito determinístico (AFD)

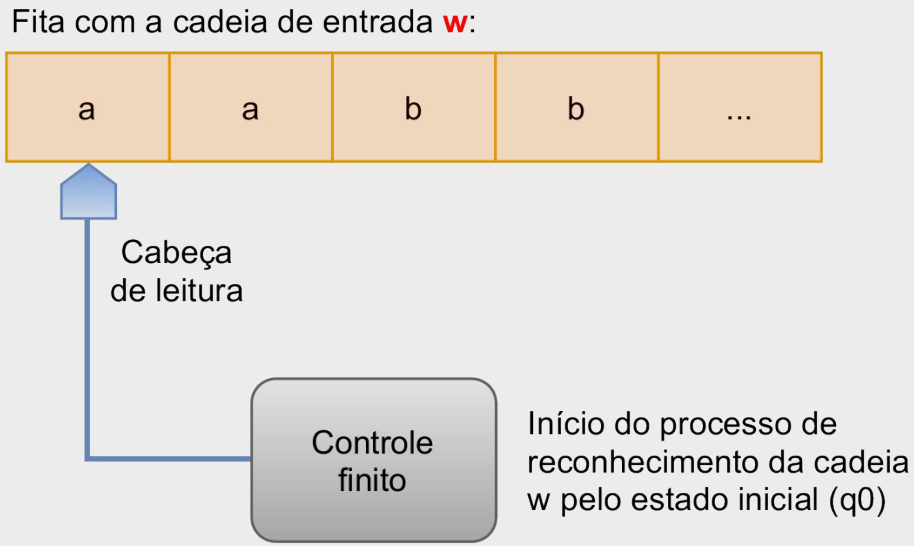
- Usaremos AFDs como **reconhecedores de linguagens**;
- Considere que o "problema" virá codificado como uma cadeia escrita em uma **fita** (de papel, por exemplo). Considere que essa fita é dividida em várias células, e que cada célula armazena somente um símbolo:



inteli

Autômato finito determinístico (AFD)

- Usaremos AFDs como **reconhecedores de linguagens**;
- Considere que o "problema" virá codificado como uma cadeia escrita em uma **fita** (de papel, por exemplo). Considere que essa fita é dividida em várias células, e que cada célula armazena somente um símbolo:



A cadeia será **aceita** pelo autômato quando:

- **Todos** os seus **símbolos** tiverem sido lidos;

  e ao mesmo tempo

- o autômato **terminou** o processamento da cadeia em um **estado final**!

Caso contrário, dizemos que a cadeia **não foi aceita (rejeitada)**.

inteli

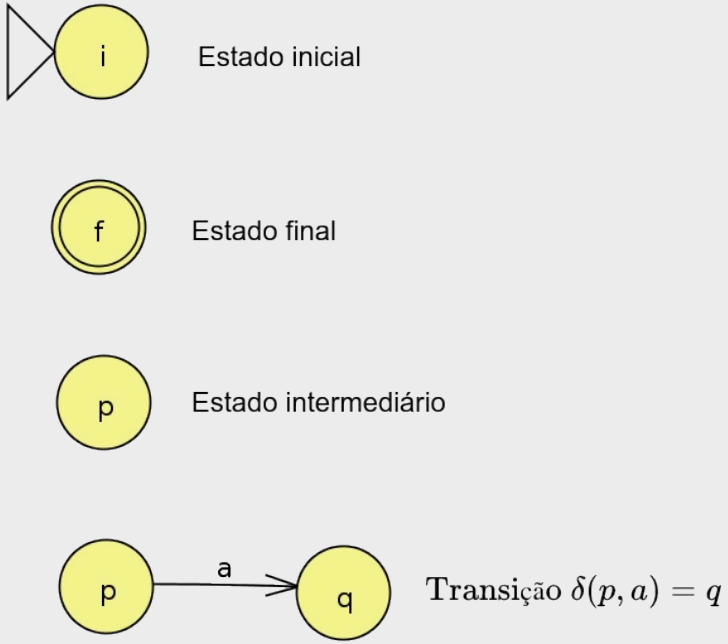
inteli



Definições sobre AFDs

Autômato finito determinístico (AFD): notação gráfica

- Existe um (único) estado onde o processamento começa, chamado **estado inicial**;
- Podem existir um ou vários **estados finais** (ou **estados de aceitação**);
- Um estado que não é inicial e nem final é dito "**intermediário**".



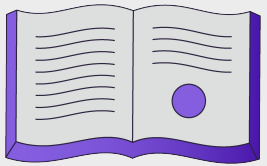
inteli

Autômato finito determinístico (AFD): definição formal

- Matematicamente, definimos um autômato finito determinístico M como uma 5-upla:

M = (Q, Σ, δ, q0, F), onde:

- **Q** é o conjunto finito de estados
- **Σ** é um conjunto finito de símbolos (alfabeto)
- **δ** é a função de transição: $Q \times \Sigma \rightarrow Q$
- **q0** é o estado inicial, tal que $q0 \in Q$
- **F** é o conjunto de estados finais, tal que $F \subseteq Q$

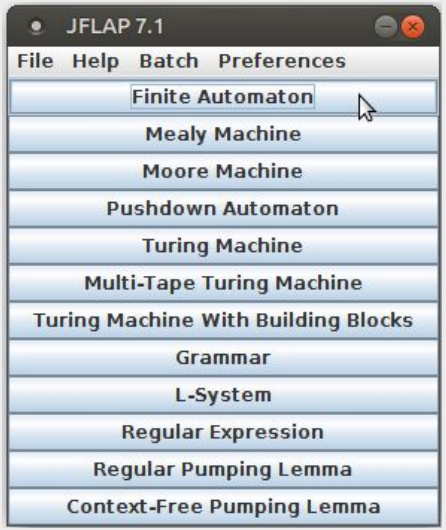


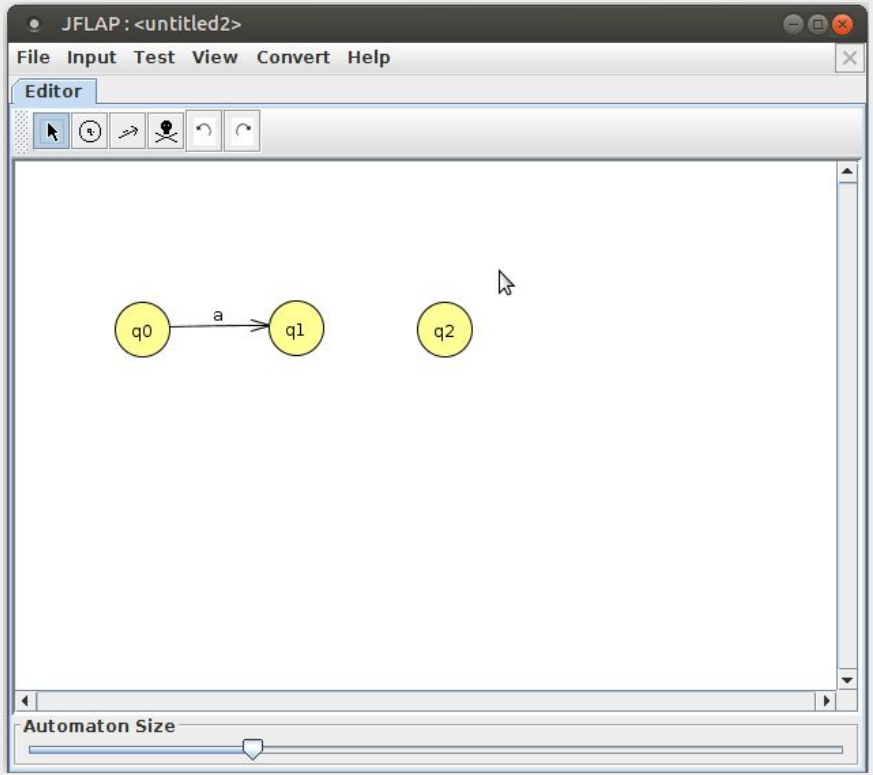
Ou seja, podemos formalizar qualquer autômato finito determinístico através destes 5 elementos!

inteli

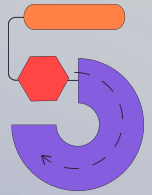
JFLAP

- **JFLAP:** software para construir e simular autômatos finitos.
  - Baixe a versão **JFLAP7.1.jar** em: https://www.jflap.org/jflaptmp/





inteli



Linguagens regulares

Linguagens regulares

- Dizemos que uma **linguagem é regular** quando ela é **reconhecida** por um **AFD**;
- De forma análoga, podemos afirmar que qualquer linguagem que um AFD reconhece é também uma linguagem regular;

    - Seja L_1 uma linguagem e M_1 um autômato: se M_1 reconhece L_1, denotamos: $\mathbf{L(M_1) = L_1}$.

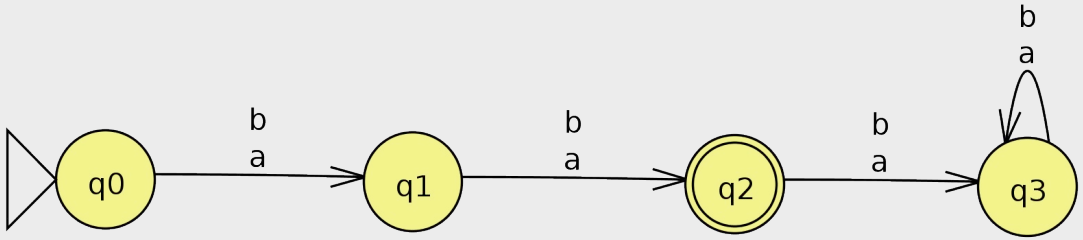
inteli

Linguagens regulares

- Dizemos que uma **linguagem é regular** quando ela é **reconhecida** por um **AFD**;
- De forma análoga, podemos afirmar que qualquer linguagem que um AFD reconhece é também uma linguagem regular;

    - Seja L_1 uma linguagem e M_1 um autômato: se M_1 reconhece L_1, denotamos: $\mathbf{L(M_1) = L_1}$.

- **Definição**: toda **linguagem finita** é **regular**!
    - Se uma linguagem possui um número finito de cadeias, é fácil imaginar que podemos construir um número finito de estados e transições para reconhecer todas as cadeias.
    - Exemplo: **L = {aa, ab, ba, bb}**



Note que o autômato é **determinístico**: para cada estado há uma (única) saída para cada símbolo do alfabeto.

inteli

Linguagens regulares vs não-regulares

- **Problema**: linguagens com **infinitas** cadeias!
  - Algumas são regulares, outras não são!

- **Conclusão**: existem linguagens sobre as quais é **impossível** construir um AFD que as reconheça!
  - Linguagens não regulares!
  - **Motivo**: limitação do modelo do AFD (especialmente limitação de memória!)

inteli

Linguagens não-regulares

- **Problema**: linguagens com **infinitas** cadeias!
  - Algumas são regulares, outras não são!

- Alguns exemplos de **linguagens não-regulares**:
  - $L_1 = \{a^n b^n \in \{a,b\}^* \mid n \geq 1\}$, seria necessário memória "ilimitada" para contar qualquer valor de n (é necessário lembrar quantos 'a' apareceram no início da cadeia);
    - ***Exemplos de cadeias:*** *ab, aabb, aaabbb, aaaabbbb, aaaaabbbbb, …*
  - $L_2 = \{a^p \in \{a\}^* \mid p \text{ é um número primo qualquer}\}$, não há um padrão de repetição nos números primos;
    - ***Exemplos de cadeias:*** *aa, aaa, aaaaa, aaaaaaa, aaaaaaaaaaa, aaaaaaaaaaaaa, …*
  - $L_3 = \{ww \mid w \in \{a,b\}^*\}$, seria necessário lembrar (memória) a primeira parte da cadeia;
    - ***Exemplos de cadeias:*** *aa, bb, abab, baba, aaaa, bbbb, abbaabba, aababaabab, …*

Sinta-se desafiado e tente construir os AFDs para reconhecer as 3 linguagens acima :-)

inteli


inteli